

Secure Coding Review

Security Audit of a Python / Flask Web Application

Task 3 - Code Review, Static Analysis, Findings and Remediation

Language: Python 3.12 | Framework: Flask | Scanner: Bandit 1.9.4 + manual inspection

Standards: OWASP Top 10 (2021), CWE, OWASP ASVS, PEP 8 / secure-coding guidelines

Table of Contents

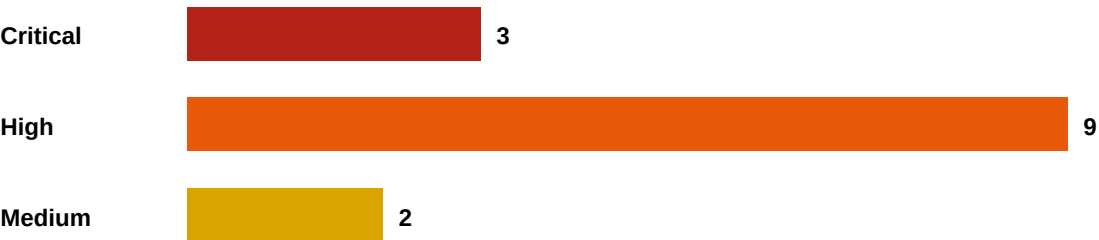
Table of Contents	2
1. Executive Summary	3
2. Objectives, Scope and Target Selection	4
3. Methodology and Tools	4
4. Static Analysis Results (Bandit)	6
5. Findings Summary and Risk Matrix	7
6. Detailed Findings and Remediation	8
7. Secure Coding Best Practices	16
8. Remediation Plan and Prioritization	18
9. Verification (Re-scan of Hardened Code)	18
10. Conclusion	20
Appendix A: Vulnerable Source Code (app.py)	21
Appendix B: Hardened Source Code (app_secure.py)	23
Appendix C: Full Bandit Output (original code)	25

1. Executive Summary

This project documents a complete secure coding review of a small Python web application built with the Flask framework. The application (**app.py**, 93 lines) offers user registration, login, file download, a network ping utility, configuration upload, a URL fetcher and an admin page. It was deliberately written with realistic, commonly seen weaknesses so that the full audit lifecycle can be demonstrated: scope definition, automated static analysis, manual code inspection, risk rating, remediation and verification.

The Bandit static analyzer reported **16 issues** (5 High, 6 Medium, 5 Low). Manual inspection then identified additional problems that tools cannot see, such as reflected template injection, path traversal, missing authorization and absent CSRF protection. In total **14 consolidated findings** are documented: 3 Critical, 9 High, 2 Medium. All were remediated in a hardened version (**app_secure.py**); a re-scan shows **0 High, 0 Medium and 1 Low** residual Bandit result.

Findings by severity (manual risk rating)



Metric	Before (app.py)	After (app_secure.py)
Lines of code scanned	68	113
Bandit High	5	0
Bandit Medium	6	0
Bandit Low	5	1
Manual findings (not detected by tool)	6	0

Overall posture before remediation: Unacceptable. An unauthenticated attacker can obtain remote code execution through at least four independent paths. The application must not be deployed until the fixes in Section 8 are applied.

2. Objectives, Scope and Target Selection

Objectives

- Select a programming language and an application to audit.
- Perform a code review to identify security vulnerabilities.
- Use static analysis tools together with manual inspection.
- Provide recommendations and secure coding best practices.
- Document findings and provide remediation steps for safer code.

Target selection and rationale

Language: Python 3 was selected because it is widely used for web back-ends, has a mature security-linting ecosystem (Bandit, Semgrep, pip-audit) and its dynamic features (pickle, eval, subprocess, string formatting) make it an excellent language for demonstrating injection and deserialization flaws.

Application: a Flask web service exposing nine HTTP routes. Flask is minimal and gives the developer full responsibility for input validation, authentication and output encoding, which is why it exposes typical secure-coding mistakes.

Route	Method	Purpose
/register	POST	Create a user account
/login	POST	Authenticate and start a session
/greet	GET	Greet a user by name
/download	GET	Serve a file from the uploads folder
/ping	GET	Run a network ping against a host
/load	POST	Load a serialized object
/config	POST	Parse a YAML configuration
/fetch	GET	Fetch a remote URL
/token, /admin	GET	Issue a token / admin panel

Scope

In scope: all Python source in app.py, its imports and runtime configuration. Out of scope: infrastructure, the operating system, third-party dependency CVEs (covered only as a recommendation) and dynamic penetration testing against production systems. Testing was performed on a local isolated copy only.

3. Methodology and Tools

The review follows a standard secure code review workflow aligned with the OWASP Code Review Guide and the OWASP ASVS verification levels.

Phase	Activity	Output
1. Recon	Read the code end to end, list entry points, trust boundaries and sensitive sinks (SQL, shell, file system, deserializers, HTTP client).	Attack-surface map
2. Automated SAST	Run Bandit 1.9.4 against the source and export JSON and text reports.	16 raw tool results
3. Manual review	Trace every user-controlled value (request.form, request.args, request.data) to its sink; check authentication, authorization, session and crypto use.	6 extra findings

Phase	Activity	Output
4. Triage	De-duplicate, map to CWE and OWASP Top 10, rate severity by exploitability and impact.	14 findings
5. Remediate	Rewrite vulnerable code using secure APIs.	app_secure.py
6. Verify	Re-run Bandit and re-inspect each fix.	Section 9 results

Tools used

Tool	Type	Role in this review
Bandit 1.9.4	Static analyzer (SAST) for Python	Pattern-based detection of insecure calls, weak crypto, hard-coded secrets, unsafe deserialization
Manual inspection	Human code review	Business logic, authorization, data-flow and context-dependent flaws (XSS/SSTI, path traversal, CSRF)
Recommended in CI	Semgrep, pip-audit, detect-secrets, OWASP ZAP	Rule-based SAST, dependency CVE scan, secret scanning, DAST (not run in this offline lab)

Command used

```

pip install bandit
bandit -r app.py -f txt -o bandit.txt
bandit -r app.py -f json -o bandit.json

```

Severity rating model

Rating	Meaning	Typical CVSS v3.1
Critical	Remote code execution or full data compromise without authentication	9.0 - 10.0
High	Serious compromise needing little effort or low privileges	7.0 - 8.9
Medium	Exploitable with preconditions, limited impact	4.0 - 6.9
Low / Info	Hardening gaps, best-practice deviations	0.1 - 3.9

CVSS values in this report are indicative estimates for teaching purposes, not formally calculated scores.

4. Static Analysis Results (Bandit)

Bandit parsed the file into an AST and ran its plug-in tests. The table lists every result exactly as reported. Note that Bandit rates a result by its own severity and confidence; the final risk ratings in Section 5 differ where context raises or lowers the real-world risk.

Line	Test ID	Issue	Sev.	Conf.	CWE
4	B403	Consider possible security implications associated with pickle module.	LOW	HIGH	CWE-502
5	B404	Consider possible security implications associated with the subprocess module.	LOW	HIGH	CWE-78
11	B105	Possible hardcoded password: 'supersecret123'	LOW	MEDIUM	CWE-259
13	B105	Possible hardcoded password: 'admin@2024'	LOW	MEDIUM	CWE-259
24	B324	Use of weak MD5 hash for security. Consider usedforsecurity=False	HIGH	HIGH	CWE-327
26	B608	Possible SQL injection vector through string-based query construction.	MEDIUM	MEDIUM	CWE-89
34	B324	Use of weak MD5 hash for security. Consider usedforsecurity=False	HIGH	HIGH	CWE-327
36	B608	Possible SQL injection vector through string-based query construction.	MEDIUM	LOW	CWE-89
59	B602	subprocess call with shell=True identified, security issue.	HIGH	HIGH	CWE-78
65	B301	Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible security ...	MEDIUM	HIGH	CWE-502
71	B506	Use of unsafe yaml load. Allows instantiation of arbitrary objects. Consider yaml.safe_load().	MEDIUM	HIGH	CWE-20
78	B501	Call to requests with verify=False disabling SSL certificate checks, security issue.	HIGH	HIGH	CWE-295
78	B113	Call to requests without timeout	MEDIUM	LOW	CWE-400
84	B311	Standard pseudo-random generators are not suitable for security/cryptographic purposes.	LOW	HIGH	CWE-330
93	B201	A Flask app appears to be run with debug=True, which exposes the Werkzeug debugger and allows the executi...	HIGH	MEDIUM	CWE-94
93	B104	Possible binding to all interfaces.	MEDIUM	MEDIUM	CWE-605

Raw scanner summary

```
Code scanned:      Total lines of code: 68   (#nosec skipped: 0)
Issues by severity:  High: 5   Medium: 6   Low: 5
Issues by confidence: High: 9   Medium: 5   Low: 2
```

Limitations of the tool

Bandit is a pattern matcher. It flagged the SQL string building (B608) only at Medium severity and did **not** detect: the template injection in /greet, the path traversal in /download, the missing authorization on /admin, missing CSRF protection or missing rate limiting. These required manual data-flow review, which demonstrates why SAST alone is never sufficient.

5. Findings Summary and Risk Matrix

ID	Finding	Severity	CVSS*	CWE	Location
F-01	SQL Injection in login	Critical	9.8	CWE-89	app.py:36
F-02	SQL Injection in registration	High	8.6	CWE-89	app.py:26
F-03	OS Command Injection in /ping	Critical	9.8	CWE-78	app.py:59
F-04	Insecure deserialization with pickle	Critical	9.8	CWE-502	app.py:65
F-05	Server-side template injection / reflected XSS in /greet	High	8.1	CWE-1336 / CWE-79	app.py:47
F-06	Path traversal in /download	High	7.5	CWE-22	app.py:53
F-07	Weak password hashing (MD5, unsalted)	High	7.5	CWE-327 / CWE-328 / CWE-916	app.py:24, 34
F-08	Hard-coded secrets	High	7.5	CWE-798 / CWE-259	app.py:11, 13
F-09	Unsafe YAML loading	High	8.1	CWE-502	app.py:71
F-10	SSRF and disabled TLS verification in /fetch	High	8.2	CWE-918 / CWE-295 / CWE-400	app.py:75-78
F-11	Debug mode enabled and binding to all interfaces	High	8.8	CWE-489 / CWE-94 / CWE-605	app.py:93
F-12	Missing authentication and authorization on /admin and other routes	High	8.6	CWE-862 / CWE-306	app.py:88 (and all routes)
F-13	Predictable token generation	Medium	5.9	CWE-338	app.py:84
F-14	Missing CSRF protection, rate limiting, secure cookies and security headers	Medium	6.5	CWE-352 / CWE-307 / CWE-614 / CWE-693	app.py (global)

*Indicative CVSS v3.1 estimate. Bandit under-rated several items: SQL injection is Medium in the tool but Critical here because the login query allows full authentication bypass.

Risk matrix (likelihood x impact)

	Low impact	Medium impact	High impact
Likely	F-13	F-14	F-01, F-03, F-04, F-11, F-12
Possible	-	F-06, F-10	F-02, F-05, F-09
Unlikely	-	-	F-07, F-08 (needs data/source leak first)

6. Detailed Findings and Remediation

F-01 SQL Injection in login			Critical
Severity / CVSS*	Critical / 9.8	CWE	CWE-89
OWASP Top 10	A03:2021 Injection	Location	app.py:36
Detected by	Bandit B608 + manual		

Description

The login query is built with an f-string that places the raw username and the password hash directly into the SQL text. Any quote character in the username changes the structure of the query.

Impact

Authentication bypass, login as any user including administrators, extraction or modification of the whole database.

Proof of concept (conceptual)

Conceptual test input for the username field: a value that closes the string and makes the WHERE clause always true (for example the classic tautology ' OR '1'='1' --). The application then returns 'Welcome' without a valid password.

Vulnerable code

```
query = f"SELECT id FROM users WHERE username='{username}' AND password='{password}'"
row = db.execute(query).fetchone()
```

Secure code

```
row = db.execute(
    "SELECT id, password, is_admin FROM users WHERE username = ?",
    (username,)
).fetchone()
if row and check_password_hash(row[1], password):
    session.clear()
    session["user_id"] = row[0]
```

Why this fix works

Parameterized queries send code and data separately, so user input can never change the query structure. The password is verified in Python, not inside SQL.

F-02 SQL Injection in registration			High
Severity / CVSS*	High / 8.6	CWE	CWE-89
OWASP Top 10	A03:2021 Injection	Location	app.py:26
Detected by	Bandit B608		

Description

The INSERT statement is created with Python % string formatting using user supplied values.

Impact

An attacker can break out of the VALUES list, insert rows with chosen data (for example admin accounts) or trigger database errors that leak schema details.

Proof of concept (conceptual)

Submitting a username containing a single quote produces a syntax error, which confirms the input reaches the SQL parser unescaped.

Vulnerable code

```
db.execute("INSERT INTO users(username, password) VALUES ('%s', '%s')" % (username, pw_hash))
```

Secure code

```
db.execute("INSERT INTO users(username, password) VALUES (?, ?)",  
          (username, pw_hash))
```

Why this fix works

Placeholders (?) let the database driver bind values safely. The same rule applies to ORMs: never concatenate into raw SQL.

F-03 OS Command Injection in /ping			Critical
Severity / CVSS*	Critical / 9.8	CWE	CWE-78
OWASP Top 10	A03:2021 Injection	Location	app.py:59
Detected by	Bandit B602 (+B404)		

Description

The host parameter is concatenated to a command string which is executed with shell=True, so the shell interprets metacharacters such as ; | && and \$().

Impact

Unauthenticated remote code execution with the privileges of the web server process: read secrets, install malware, pivot to internal network.

Proof of concept (conceptual)

A value such as 8.8.8.8; id in the host parameter makes the shell run the second command after ping. (Shown for explanation only; test on isolated lab systems.)

Vulnerable code

```
host = request.args.get("host")  
output = subprocess.check_output("ping -c 1 " + host, shell=True)
```

Secure code

```
HOST_RE = re.compile(r"^[A-Za-z0-9]([A-Za-z0-9.-]{0,251}[A-Za-z0-9])?$")  
if not HOST_RE.match(host):  
    abort(400)  
out = subprocess.run(["ping", "-c", "1", host], capture_output=True,  
                    text=True, timeout=5, check=False)
```

Why this fix works

Passing an argument list without a shell removes shell parsing entirely. The allow-list regex rejects spaces and metacharacters, a timeout prevents hanging, and the route is restricted to administrators.

F-04 Insecure deserialization with pickle			Critical
Severity / CVSS*	Critical / 9.8	CWE	CWE-502
OWASP Top 10	A08:2021 Software and Data Integrity Failures	Location	app.py:65
Detected by	Bandit B301 (+B403)		

Description

pickle.loads() is called on the raw request body. Pickle can execute arbitrary Python code while loading, via the __reduce__ method of a crafted object.

Impact

Unauthenticated remote code execution; the attacker needs only to send a crafted payload.

Proof of concept (conceptual)

Any object whose `__reduce__` returns a callable (for example `os.system` with an argument) is executed at load time. No exploit code is reproduced here.

Vulnerable code

```
data = pickle.loads(request.data)
return str(data)
```

Secure code

```
return jsonify(json.loads(request.data)) # data-only format
# validate against a schema (e.g. jsonschema / pydantic) before use
```

Why this fix works

JSON can only represent data, never code. Never unpickle data from the network, files or caches that an attacker could influence; if unavoidable, sign data with HMAC and verify first.

F-05 Server-side template injection / reflected XSS in /greet			High
Severity / CVSS*	High / 8.1	CWE	CWE-1336 / CWE-79
OWASP Top 10	A03:2021 Injection	Location	app.py:47
Detected by	Manual only		

Description

User input is concatenated into a template string which is then rendered by Jinja2 via `render_template_string`. The input becomes part of the template source, not just data.

Impact

Reflected XSS (script runs in victim's browser) and, with Jinja2 expression syntax, server-side template evaluation that can lead to data disclosure or code execution.

Proof of concept (conceptual)

Requesting `/greet?name={{7*7}}` returns 49 instead of the literal text, proving the expression was evaluated by the server. A name containing a script tag also executes in the browser.

Vulnerable code

```
return render_template_string("<h1>Hello " + name + "</h1>")
```

Secure code

```
from markupsafe import escape
return "<h1>Hello {}</h1>".format(escape(name))
# or: render_template("greet.html", name=name) (auto-escaped)
```

Why this fix works

Treat user input only as template variables, never as template text. Jinja2 file templates auto-escape HTML by default.

F-06 Path traversal in /download			High
Severity / CVSS*	High / 7.5	CWE	CWE-22
OWASP Top 10	A01:2021 Broken Access Control	Location	app.py:53
Detected by	Manual only		

Description

The filename query parameter is joined to the uploads folder with `os.path.join` without validation. Sequences such as `../` escape the folder, and an absolute path replaces it completely.

Impact

Disclosure of arbitrary files readable by the application, including source code, configuration, the database file and system files.

Proof of concept (conceptual)

/download?file=../app.py returns the application source; /download?file=/etc/passwd shows how os.path.join discards the base directory when given an absolute path.

Vulnerable code

```
filename = request.args.get("file")
return send_file(os.path.join("uploads", filename))
```

Secure code

```
from flask import send_from_directory
return send_from_directory(UPLOAD_DIR, filename) # uses safe_join, 404 on escape
```

Why this fix works

send_from_directory resolves the final path and refuses anything outside the base directory. Also require authentication and use opaque IDs instead of raw file names where possible.

F-07 Weak password hashing (MD5, unsalted)			High
Severity / CVSS*	High / 7.5	CWE	CWE-327 / CWE-328 / CWE-916
OWASP Top 10	A02:2021 Cryptographic Failures	Location	app.py:24, 34
Detected by	Bandit B324		

Description

Passwords are stored as plain MD5 digests. MD5 is fast, collision-prone and unsalted, so identical passwords produce identical hashes.

Impact

After any database leak (see SQL injection) most passwords can be recovered in seconds using rainbow tables or GPU cracking.

Proof of concept (conceptual)

Not applicable - offline attack after data theft. Common passwords hash to well known values that can be looked up.

Vulnerable code

```
pw_hash = hashlib.md5(password.encode()).hexdigest()
```

Secure code

```
from werkzeug.security import generate_password_hash, check_password_hash
pw_hash = generate_password_hash(password) # scrypt, per-user salt
check_password_hash(stored_hash, password) # constant-time compare
```

Why this fix works

Use a slow, salted, memory-hard password hashing function (Argon2id, scrypt, bcrypt or PBKDF2 with high iterations). Enforce a minimum length (12+) and check against breached-password lists.

F-08 Hard-coded secrets			High
Severity / CVSS*	High / 7.5	CWE	CWE-798 / CWE-259
OWASP Top 10	A07:2021 Identification and Authentication Failures	Location	app.py:11, 13
Detected by	Bandit B105		

Description

The Flask secret key and an admin password are written in source code. They end up in version control, backups and developer machines.

Impact

Anyone with read access to the repository can forge valid session cookies and impersonate any user, including administrators.

Proof of concept (conceptual)

With the known SECRET_KEY an attacker can sign a session cookie containing any user_id using standard Flask tooling.

Vulnerable code

```
app.secret_key = "supersecret123"
ADMIN_PASSWORD = "admin@2024"
```

Secure code

```
SECRET_KEY = os.environ["SECRET_KEY"]      # generated with secrets.token_hex(32)
# remove ADMIN_PASSWORD; create admins through a hashed, audited process
```

Why this fix works

Load secrets from environment variables or a secrets manager (Vault, AWS Secrets Manager), rotate them, and add detect-secrets or git-secrets to pre-commit. Rotate any value that was ever committed.

F-09 Unsafe YAML loading			High
Severity / CVSS*	High / 8.1	CWE	CWE-502
OWASP Top 10	A08:2021 Software and Data Integrity Failures	Location	app.py:71
Detected by	Bandit B506		

Description

yaml.load() without a safe Loader can construct arbitrary Python objects from tags in the document.

Impact

Remote code execution through a malicious YAML upload (similar to pickle).

Proof of concept (conceptual)

A YAML document using Python object tags can instantiate classes or call functions during parsing. Payload intentionally omitted.

Vulnerable code

```
cfg = yaml.load(request.data)
```

Secure code

```
cfg = yaml.safe_load(request.data)
```

Why this fix works

safe_load only builds basic types (dict, list, str, int). Validate the resulting structure against a schema.

F-10 SSRF and disabled TLS verification in /fetch			High
Severity / CVSS*	High / 8.2	CWE	CWE-918 / CWE-295 / CWE-400
OWASP Top 10	A10:2021 SSRF	Location	app.py:75-78
Detected by	Bandit B501, B113 + manual		

Description

The server fetches any URL supplied by the user, with certificate verification turned off and no timeout.

Impact

Attackers can make the server call internal services or cloud metadata endpoints (SSRF), read internal data, scan the network, and man-in-the-middle attacks become trivial. Missing timeout allows resource exhaustion.

Proof of concept (conceptual)

/fetch?url=http://169.254.169.254/ (cloud metadata) or http://localhost:6379/ targets services only reachable from the server.

Vulnerable code

```
url = request.args.get("url")
return requests.get(url, verify=False).text
```

Secure code

```
url = urlparse(request.args.get("url", ""))
if url.scheme != "https" or url.hostname not in ALLOWED_FETCH_HOSTS:
    abort(400)
return requests.get(url.geturl(), timeout=5, allow_redirects=False).text
```

Why this fix works

Use an allow-list of destinations, HTTPS only, keep certificate verification enabled (default), set timeouts, block redirects and private IP ranges, and run egress filtering at network level.

F-11 Debug mode enabled and binding to all interfaces			High
Severity / CVSS*	High / 8.8	CWE	CWE-489 / CWE-94 / CWE-605
OWASP Top 10	A05:2021 Security Misconfiguration	Location	app.py:93
Detected by	Bandit B201, B104		

Description

app.run(debug=True) exposes the interactive Werkzeug debugger, which allows code execution from the browser, and host 0.0.0.0 makes it reachable from every network interface.

Impact

Remote code execution via the debugger console and leakage of source code and environment variables through stack traces.

Proof of concept (conceptual)

Triggering any exception shows a traceback page with an interactive console (protected only by a PIN that is derivable in some situations).

Vulnerable code

```
app.run(host="0.0.0.0", port=5000, debug=True)
```

Secure code

```
# development only, bound to localhost
app.run(host="127.0.0.1", port=5000, debug=False)
# production: gunicorn behind a reverse proxy (nginx) with TLS
```

Why this fix works

Never ship debug mode. Drive configuration from environment, use a production WSGI server, and bind to the narrowest interface needed.

F-12 Missing authentication and authorization on /admin and other routes			High
Severity / CVSS*	High / 8.6	CWE	CWE-862 / CWE-306
OWASP Top 10	A01:2021 Broken Access Control	Location	app.py:88 (and all routes)
Detected by	Manual only		

Description

The admin route performs no check of who the caller is. Sensitive routes (/download, /load, /config, /fetch, /ping) are also open to anonymous users.

Impact

Anyone can reach administrative functionality and every dangerous feature, which multiplies the effect of all other findings.

Proof of concept (conceptual)

A plain unauthenticated GET /admin returns the admin panel text.

Vulnerable code

```
@app.route("/admin")
def admin():
    return "Admin panel: all users data"
```

Secure code

```
def login_required(admin=False):
    def deco(fn):
        @wraps(fn)
        def wrapper(*a, **kw):
            if "user_id" not in session: abort(401)
            if admin and not session.get("is_admin"): abort(403)
            return fn(*a, **kw)
        return wrapper
    return deco

@app.route("/admin")
@login_required(admin=True)
def admin(): ...
```

Why this fix works

Deny by default: every route needs an explicit authentication and role check on the server. Never rely on hiding URLs.

F-13 Predictable token generation			Medium
Severity / CVSS*	Medium / 5.9	CWE	CWE-338
OWASP Top 10	A02:2021 Cryptographic Failures	Location	app.py:84
Detected by	Bandit B311		

Description

The token endpoint uses random.randint, a Mersenne Twister generator that is not cryptographically secure; outputs can be predicted after observing enough values. The value space is also only 900,000.

Impact

Tokens intended for password resets or API access can be guessed or brute-forced.

Proof of concept (conceptual)

Observing a few hundred outputs allows state recovery; even without that, 900,000 values can be enumerated quickly.

Vulnerable code

```
return str(random.randint(100000, 999999))
```

Secure code

```
import secrets
return secrets.token_urlsafe(16)    # 128 bits from the OS CSPRNG
```

Why this fix works

Use the secrets module (or os.urandom) for anything security related; random is for simulations and games.

F-14 Missing CSRF protection, rate limiting, secure cookies and security headers			Medium
Severity / CVSS*	Medium / 6.5	CWE	CWE-352 / CWE-307 / CWE-614 / CWE-693
OWASP Top 10	A05:2021 Security Misconfiguration / A07	Location	app.py (global)
Detected by	Manual only		

Description

State-changing POST routes accept requests with no anti-CSRF token; login has no brute-force throttling; session cookies are not flagged Secure, HttpOnly or SameSite; no security headers are sent; the session is not regenerated after login.

Impact

Cross-site request forgery, credential stuffing and brute force, session theft over HTTP or via XSS, clickjacking and MIME sniffing.

Proof of concept (conceptual)

A malicious web page can auto-submit a form to /register or /config while the victim is logged in.

Vulnerable code

```
# no protection configured
app = Flask(__name__)
```

Secure code

```
app.config.update(SESSION_COOKIE_HTTPONLY=True, SESSION_COOKIE_SECURE=True,
                  SESSION_COOKIE_SAMESITE="Lax", MAX_CONTENT_LENGTH=1024*1024)
@app.after_request
def security_headers(r):
    r.headers["X-Content-Type-Options"] = "nosniff"
    r.headers["X-Frame-Options"] = "DENY"
    r.headers["Content-Security-Policy"] = "default-src 'self'"
    r.headers["Strict-Transport-Security"] = "max-age=31536000"
    return r
# also: Flask-WTF CsrfProtect, Flask-Limiter on /login, session.clear() on login
```

Why this fix works

Defense in depth. The code shows cookie flags and headers; CSRF tokens (Flask-WTF) and rate limiting (Flask-Limiter) are recommended libraries to add in the framework configuration.

7. Secure Coding Best Practices

The findings above fall into a small number of recurring root causes. The practices below address each of them and map to the OWASP Top 10 and Secure Coding Practices checklist.

Area	Practice	Prevents
Input handling	Treat all external data (forms, query string, headers, cookies, files, YAML/JSON bodies) as untrusted. Validate by allow-list (type, length, format), reject rather than sanitize.	F-03, F-05, F-06
Database access	Always use parameterized queries or an ORM. Use a least-privilege database account (no DROP, no file access).	F-01, F-02
Output encoding	Auto-escape by default; use file templates; add a strict Content-Security-Policy.	F-05
Command execution	Avoid shells. Use subprocess with an argument list, shell=False, timeouts and allow-listed arguments, or use a native library instead of a command.	F-03
Deserialization	Never use pickle, marshal or yaml.load on untrusted data. Use JSON with schema validation; sign serialized data when integrity matters.	F-04, F-09
Authentication	Hash with Argon2id/scrypt/bcrypt; enforce MFA, lockout and rate limits; regenerate sessions at login; generic error messages.	F-07, F-14
Authorization	Deny by default; enforce checks on the server for every request; test horizontal and vertical privilege escalation.	F-12
Secrets management	No secrets in code; use environment/secret manager; rotate; scan commits with detect-secrets, gitleaks.	F-08
Cryptography	Use vetted libraries and modern algorithms; secrets module for randomness; TLS everywhere with verification on.	F-07, F-10, F-13
File handling	Never build paths from user input; use send_from_directory/safe_join; store uploads outside web root; restrict extensions and size.	F-06
Network requests	Allow-list outbound destinations; block private IP ranges; set timeouts; disable redirects.	F-10
Configuration	Debug off, minimal bind address, secure cookies, security headers, least privilege process user, container hardening.	F-11, F-14
Error handling & logging	Show generic errors to users; log security events (failed logins, authorization failures) without secrets; monitor and alert.	F-11
Dependencies	Pin versions, run pip-audit/Dependabot, remove unused packages, verify package integrity.	Supply chain
Process (SDLC)	Threat modeling, peer review with a security checklist, SAST and secret scanning in CI, DAST before release, security training.	All

Secure code review checklist

- Are all SQL statements parameterized?
- Does any user value reach subprocess, eval, exec, os.system, pickle, yaml.load or a template string?
- Is every route authenticated and authorized on the server side?
- Are passwords hashed with a modern salted algorithm?
- Are secrets, keys and tokens kept out of source control?
- Is randomness for security produced by the secrets module?
- Are file paths, URLs and hostnames validated against an allow-list?
- Is debug disabled and are security headers and cookie flags set?
- Are dependencies scanned and up to date?

- Do tests cover negative and abuse cases (injection strings, traversal, forced browsing)?

8. Remediation Plan and Prioritization

Priority	Timeline	Actions	Findings
P0 - Immediate	Before any deployment (0-2 days)	Disable debug and bind to localhost; remove pickle and unsafe YAML; replace shell command with argument list; parameterize SQL; add authentication to every route; rotate the exposed secret key.	F-01, F-02, F-03, F-04, F-09, F-11, F-12, F-08
P1 - Short term	1 week	Fix template injection and path traversal; migrate password storage to scrypt/Argon2 with forced reset; SSRF allow-list and TLS verification.	F-05, F-06, F-07, F-10
P2 - Medium term	2-4 weeks	CSRF tokens, rate limiting, secure cookies and headers, replace random with secrets, input schemas, central logging.	F-13, F-14
P3 - Continuous	Ongoing	Bandit/Semgrep and secret scanning in CI, dependency audits, security unit tests, annual external penetration test, developer training.	All

Migration note for existing users

Existing MD5 hashes cannot be converted directly. Recommended approach: on next successful login verify against MD5 once, immediately re-hash the plain password with scrypt and store it, then delete the legacy hash. Users who do not log in within a set period should be forced to reset their password. Because the secret key was exposed, all existing sessions must be invalidated by rotating it.

9. Verification (Re-scan of Hardened Code)

After remediation the secured version **app_secure.py** was scanned with the same Bandit command. The comparison below shows the effect of the fixes.

Test	Before	After
Total issues	16	1
High severity	5	0
Medium severity	6	0
Low severity	5	1

Residual result

One Low result remains: **B607** (line 104), 'starting a process with a partial executable path'. The call is `subprocess.run(["ping", ...])`. This is an accepted risk: the argument list is fixed, the host is validated, the route is admin-only and `shell=False`. It can be removed by using the absolute path `/bin/ping` or the native `ping3` library. The B603 notice for subprocess without shell was reviewed and suppressed with an inline `nosec` comment because the input is validated.

Manual re-test checklist

Test case	Expected result with fixed code
Username ' OR '1'='1' -- on /login	401 Invalid credentials (treated as a literal name)
/ping?host=8.8.8.8;id	400 Bad Request (regex rejects ';')
/greet?name={{7*7}}	Page shows the literal text <code>{{7*7}}</code>
/download?file=../app.py	404/NotFound (safe_join blocks traversal)
POST pickle payload to /load	JSON decoding error, no code execution

Test case	Expected result with fixed code
/fetch?url=http://169.254.169.254/	400 Bad Request (not https / not allow-listed)
GET /admin without login / as normal user	401 Unauthorized / 403 Forbidden
Start the app without SECRET_KEY	Application refuses to start (KeyError)

Static analysis cannot prove absence of vulnerabilities. A complete assurance programme would add unit tests for each abuse case above, dynamic scanning with OWASP ZAP or Burp Suite, and a dependency scan with pip-audit.

10. Conclusion

The audit demonstrates that a short application can contain a large number of serious defects, and that most of them follow from a single habit: trusting user input and using convenient but unsafe APIs (string-built SQL, shell=True, pickle, yaml.load, render_template_string). Combining automated static analysis with manual review found 14 distinct issues, three of them Critical and several enabling unauthenticated remote code execution. Replacing these constructs with safe-by-design alternatives eliminated every High and Medium Bandit result and closed the manual findings.

Key lessons

- Use tools early and often, but never rely on them alone: six findings were invisible to Bandit.
- Prefer APIs that are safe by construction (parameter binding, argument lists, JSON, auto-escaping) over defensive filtering.
- Apply deny-by-default authorization and least privilege everywhere.
- Keep secrets out of code and configuration in version control.
- Automate security checks in CI so regressions are caught before release.

References

- OWASP Top 10 (2021) - owasp.org/Top10
- OWASP Application Security Verification Standard (ASVS) 4.0
- OWASP Code Review Guide and Secure Coding Practices Quick Reference
- MITRE Common Weakness Enumeration (CWE) - cwe.mitre.org
- Bandit documentation - bandit.readthedocs.io
- Flask Security Considerations - flask.palletsprojects.com
- Python documentation: subprocess, secrets, hashlib security notes
- NIST SP 800-63B Digital Identity Guidelines (password storage)

Appendix A: Vulnerable Source Code (app.py)

```
1 import os
2 import sqlite3
3 import hashlib
4 import pickle
5 import subprocess
6 import yaml
7 import requests
8 from flask import Flask, request, render_template_string, send_file, session
9
10 app = Flask(__name__)
11 app.secret_key = "supersecret123"
12 DB = "users.db"
13 ADMIN_PASSWORD = "admin@2024"
14
15
16 def get_db():
17     return sqlite3.connect(DB)
18
19
20 @app.route("/register", methods=["POST"])
21 def register():
22     username = request.form["username"]
23     password = request.form["password"]
24     pw_hash = hashlib.md5(password.encode()).hexdigest()
25     db = get_db()
26     db.execute("INSERT INTO users(username, password) VALUES ('%s', '%s')" % (username, pw_hash))
27     db.commit()
28     return "Registered"
29
30
31 @app.route("/login", methods=["POST"])
32 def login():
33     username = request.form["username"]
34     password = hashlib.md5(request.form["password"].encode()).hexdigest()
35     db = get_db()
36     query = f"SELECT id FROM users WHERE username='{username}' AND password='{password}'"
37     row = db.execute(query).fetchone()
38     if row:
39         session["user_id"] = row[0]
40         return "Welcome"
41     return "Invalid credentials", 401
42
43
44 @app.route("/greet")
45 def greet():
46     name = request.args.get("name", "guest")
47     return render_template_string("<h1>Hello " + name + "</h1>")
48
49
50 @app.route("/download")
51 def download():
52     filename = request.args.get("file")
53     return send_file(os.path.join("uploads", filename))
54
55
56 @app.route("/ping")
57 def ping():
58     host = request.args.get("host")
59     output = subprocess.check_output("ping -c 1 " + host, shell=True)
60     return output
61
62
63 @app.route("/load", methods=["POST"])
64 def load():
65     data = pickle.loads(request.data)
66     return str(data)
67
68
69 @app.route("/config", methods=["POST"])
70 def config():
71     cfg = yaml.load(request.data)
72     return str(cfg)
73
74
75 @app.route("/fetch")
76 def fetch():
77     url = request.args.get("url")
```

```
78     return requests.get(url, verify=False).text
79
80
81 @app.route("/token")
82 def token():
83     import random
84     return str(random.randint(100000, 999999))
85
86
87 @app.route("/admin")
88 def admin():
89     return "Admin panel: all users data"
90
91
92 if __name__ == "__main__":
93     app.run(host="0.0.0.0", port=5000, debug=True)
```

Appendix B: Hardened Source Code (app_secure.py)

```
1 import os
2 import re
3 import json
4 import secrets
5 import sqlite3
6 import subprocess # nsec B404
7 from functools import wraps
8 from urllib.parse import urlparse
9
10 import requests
11 import yaml
12 from flask import (Flask, request, session, abort, jsonify,
13                   send_from_directory)
14 from markupsafe import escape
15 from werkzeug.security import generate_password_hash, check_password_hash
16
17 app = Flask(__name__)
18 app.config.update(
19     SECRET_KEY=os.environ["SECRET_KEY"],          # injected, never committed
20     SESSION_COOKIE_HTTPONLY=True,
21     SESSION_COOKIE_SECURE=True,
22     SESSION_COOKIE_SAMESITE="Lax",
23     MAX_CONTENT_LENGTH=1 * 1024 * 1024,          # 1 MB request cap
24 )
25 DB = os.environ.get("DB_PATH", "users.db")
26 UPLOAD_DIR = os.path.abspath("uploads")
27 ALLOWED_FETCH_HOSTS = {"api.example.com"}
28 HOST_RE = re.compile(r"^[A-Za-z0-9]([A-Za-z0-9.-]{0,251}[A-Za-z0-9])?$")
29
30
31 def get_db():
32     return sqlite3.connect(DB)
33
34
35 def login_required(admin=False):
36     def deco(fn):
37         @wraps(fn)
38         def wrapper(*a, **kw):
39             if "user_id" not in session:
40                 abort(401)
41             if admin and not session.get("is_admin"):
42                 abort(403)
43             return fn(*a, **kw)
44         return wrapper
45     return deco
46
47
48 @app.after_request
49 def security_headers(resp):
50     resp.headers["X-Content-Type-Options"] = "nosniff"
51     resp.headers["X-Frame-Options"] = "DENY"
52     resp.headers["Content-Security-Policy"] = "default-src 'self'"
53     resp.headers["Strict-Transport-Security"] = "max-age=31536000"
54     return resp
55
56
57 @app.route("/register", methods=["POST"])
58 def register():
59     username = request.form["username"]
60     password = request.form["password"]
61     if len(password) < 12:
62         abort(400, "Password must be at least 12 characters")
63     pw_hash = generate_password_hash(password)          # scrypt + salt
64     db = get_db()
65     db.execute("INSERT INTO users(username, password) VALUES (?, ?)",
66               (username, pw_hash))
67     db.commit()
68     return "Registered"
69
70
71 @app.route("/login", methods=["POST"])
72 def login():
73     username = request.form["username"]
74     password = request.form["password"]
75     db = get_db()
76     row = db.execute("SELECT id, password, is_admin FROM users "
77                     "WHERE username = ?", (username,)).fetchone()
```

```

78     if row and check_password_hash(row[1], password):
79         session.clear()
80         session["user_id"], session["is_admin"] = row[0], bool(row[2])
81         return "welcome"
82     return "Invalid credentials", 401
83
84
85 @app.route("/greet")
86 def greet():
87     name = request.args.get("name", "guest")
88     return "<h1>Hello {}</h1>".format(escape(name))
89
90
91 @app.route("/download")
92 @login_required()
93 def download():
94     filename = request.args.get("file", "")
95     return send_from_directory(UPLOAD_DIR, filename)    # safe_join inside
96
97
98 @app.route("/ping")
99 @login_required(admin=True)
100 def ping():
101     host = request.args.get("host", "")
102     if not HOST_RE.match(host):
103         abort(400)
104     out = subprocess.run(["ping", "-c", "1", host], capture_output=True,
105                         text=True, timeout=5, check=False) # nsec B603
106     return out.stdout
107
108
109 @app.route("/load", methods=["POST"])
110 @login_required()
111 def load():
112     return jsonify(json.loads(request.data))            # data only, no code
113
114
115 @app.route("/config", methods=["POST"])
116 @login_required()
117 def config():
118     return jsonify(yaml.safe_load(request.data))
119
120
121 @app.route("/fetch")
122 @login_required()
123 def fetch():
124     url = urlparse(request.args.get("url", ""))
125     if url.scheme != "https" or url.hostname not in ALLOWED_FETCH_HOSTS:
126         abort(400)
127     return requests.get(url.geturl(), timeout=5, allow_redirects=False).text
128
129
130 @app.route("/token")
131 @login_required()
132 def token():
133     return secrets.token_urlsafe(16)
134
135
136 @app.route("/admin")
137 @login_required(admin=True)
138 def admin():
139     return "Admin panel"
140
141
142 if __name__ == "__main__":
143     app.run(host="127.0.0.1", port=5000, debug=False)

```

Appendix C: Full Bandit Output (original code)

```
>> Issue: [B403:blacklist] Severity: LOW Confidence: HIGH
Consider possible security implications associated with pickle module.
CWE-502 Location: app.py:4
-----
>> Issue: [B404:blacklist] Severity: LOW Confidence: HIGH
Consider possible security implications associated with the subprocess module.
CWE-78 Location: app.py:5
-----
>> Issue: [B105:hardcoded_password_string] Severity: LOW Confidence: MEDIUM
Possible hardcoded password: 'supersecret123'
CWE-259 Location: app.py:11
-----
>> Issue: [B105:hardcoded_password_string] Severity: LOW Confidence: MEDIUM
Possible hardcoded password: 'admin@2024'
CWE-259 Location: app.py:13
-----
>> Issue: [B324:hashlib] Severity: HIGH Confidence: HIGH
Use of weak MD5 hash for security. Consider usedforsecurity=False
CWE-327 Location: app.py:24
-----
>> Issue: [B608:hardcoded_sql_expressions] Severity: MEDIUM Confidence: MEDIUM
Possible SQL injection vector through string-based query construction.
CWE-89 Location: app.py:26
-----
>> Issue: [B324:hashlib] Severity: HIGH Confidence: HIGH
Use of weak MD5 hash for security. Consider usedforsecurity=False
CWE-327 Location: app.py:34
-----
>> Issue: [B608:hardcoded_sql_expressions] Severity: MEDIUM Confidence: LOW
Possible SQL injection vector through string-based query construction.
CWE-89 Location: app.py:36
-----
>> Issue: [B602:subprocess_popen_with_shell_equals_true] Severity: HIGH Confidence: HIGH
subprocess call with shell=True identified, security issue.
CWE-78 Location: app.py:59
-----
>> Issue: [B301:blacklist] Severity: MEDIUM Confidence: HIGH
Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data,
possible security issue.
CWE-502 Location: app.py:65
-----
>> Issue: [B506:yaml_load] Severity: MEDIUM Confidence: HIGH
Use of unsafe yaml load. Allows instantiation of arbitrary objects. Consider
yaml.safe_load().
CWE-20 Location: app.py:71
-----
>> Issue: [B501:request_with_no_cert_validation] Severity: HIGH Confidence: HIGH
Call to requests with verify=False disabling SSL certificate checks, security issue.
CWE-295 Location: app.py:78
-----
>> Issue: [B113:request_without_timeout] Severity: MEDIUM Confidence: LOW
Call to requests without timeout
CWE-400 Location: app.py:78
-----
>> Issue: [B311:blacklist] Severity: LOW Confidence: HIGH
Standard pseudo-random generators are not suitable for security/cryptographic purposes.
CWE-330 Location: app.py:84
-----
>> Issue: [B201:flask_debug_true] Severity: HIGH Confidence: MEDIUM
A Flask app appears to be run with debug=True, which exposes the Werkzeug debugger and
allows the execution of arbitrary code.
CWE-94 Location: app.py:93
-----
>> Issue: [B104:hardcoded_bind_all_interfaces] Severity: MEDIUM Confidence: MEDIUM
Possible binding to all interfaces.
CWE-605 Location: app.py:93
-----
```